`auto`: Initialiser-Driven Type Deduction in Selfie

Initialiser-driven type deduction in a single-pass C* compiler without an AST

Florian Schroffner
florian.schroffner@stud.plus.ac.at
Department of Computer Sciences, University of Salzburg
Advanced Systems Engineering
Supervisor: Univ.-Prof. Dr. Christoph Kirsch

July 8, 2026

Abstract

This work extends the Selfie/C* compiler with a single keyword, `auto`, that gives a local variable the type of its initialiser. The feature is added *without* changing Selfie's single-pass architecture: no abstract syntax tree (AST), no separate semantic pass, and no change to the code generator. The enabling observation is that Selfie's parser already performs lightweight type tracking — every expression-compiling function *returns* the type (`uint64_t` or `uint64_t*`) of the value it just emitted — so the type an `auto` variable needs is already computed the moment its initialiser is parsed. We distinguish the mechanism we add, *type deduction* (local, one-directional, as in C++ and Go), from full *type inference* (global, constraint-solving, as in the ML family), and argue that it is precisely deduction's locality that makes it fit a single pass. The implementation touches three functions in `selfie.c` plus a one-keyword scanner addition. The extended compiler still self-compiles to a 43 624-instruction RISC-U binary and passes the project autograder, including a dedicated `auto`-type-inference suite. As a self-application test, 79 of `selfie.c`'s own local declarations are rewritten cast-free to `auto`; the compiler translates this version of itself and still reaches a self-self fixpoint. The point is not that initialiser-driven `auto` is hard — C++ and Go deduce this way in a single pass too — but that in Selfie the intervention is nearly free: the type an `auto` variable needs is a by-product the single pass already computes and would otherwise discard, so capturing it costs three functions and no new machinery.

Keywords: Type Deduction, Type Inference, `auto`, Single-Pass Compiler, Recursive-Descent Parsing, Selfie, C*, RISC-U, Self-Compilation

1 Introduction

1.1 Problem Definition

In C and most C-family languages every local variable is declared with an explicit type, even when that type is obvious from the value it is initialised with. Modern languages remove this redundancy: C++ offers `auto`, Go provides `:=`, and the ML family infers types across whole functions. A common assumption is that adding such a feature to a compiler requires a multi-pass pipeline with an explicit AST and a dedicated semantic phase that can reason about types before code generation. Selfie's compiler has neither: it is single-pass and emits machine code as it parses. The problem this work addresses is therefore whether initialiser-driven `auto` — a form of type *deduction*, which we distinguish from full type inference in Section 1.3.1 — can be added to Selfie *without* abandoning its defining single-pass, AST-free design.

1.2 Contribution

We design and implement the `auto` keyword for local variables in C*, the language compiled by Selfie. A variable declared `auto` takes the type of its initialiser. The feature is realised entirely within Selfie's single pass by reading back type information the parser already produces, so it adds no AST and no separate pass. We touch three functions in `selfie.c`. The extended compiler still self-compiles and passes the autograder. In addition, we dogfood the feature on the compiler itself: 79 of `selfie.c`'s own local

declarations are folded into `auto` cast-free, and the resulting compiler still reaches a self-self fixpoint. Finally, we analyse why the mechanism is cheap by separating *type deduction* from *type inference*, and we relate the feature's restrictions to the boundary of what a single pass makes available.

This work answers three research questions:

RQ1. *Feasibility.* Can initialiser-driven `auto` deduction be added to a single-pass compiler without introducing an AST or a separate semantic pass?

RQ2. *Mechanism.* By what mechanism are the two open problems — reading back the type and allocating the variable's stack slot in a single pass — solved, and how localised is the resulting change to `selfie.c`?

RQ3. *Self-referentiality.* Does the resulting compiler still satisfy Selfie's requirement that it compile its own source, even when that source itself uses `auto`?

1.3 Type Inference Primer

1.3.1 Type Deduction versus Type Inference

The phrase "type inference" is used loosely for two mechanisms that differ sharply in cost and power. Telling them apart is essential to understanding what this work does — and what it deliberately does not.

Type deduction (local, one-directional). Given a variable whose initialiser is a *fully typed* expression, the variable simply adopts that expression's type. Information flows in one direction only — from the right-hand side of the = to the variable on the left — and there is nothing to *solve*: the type of the initialiser is already known the moment the declaration is read. This is what C++ does — the ISO standard literally calls it "`auto` type *deduction*" [3] — what Go's `:=` does, and what C23 `auto` [4] performs. A deducer never reasons about how the variable is *used* afterwards; it inspects only the single expression to its right.

Type inference (global, bidirectional). Languages of the ML family — Haskell, OCaml, Standard ML — assign types to variables and functions that carry *no* annotation on either side. They do so by collecting *constraints* from every use across a definition (or the whole program) and *solving* them by unification, the Hindley–Milner algorithm [2]. Type information may flow *backward*, from a later use to an earlier definition, and a value may be *generalised* to a polymorphic type. In `let n = read () in n + 1`, an inferrer can fix the type of `n` from the way it is later used; a deducer cannot, because it has already moved past the declaration.

Why the distinction is architectural. Hindley–Milner inference needs a structure that survives long enough to gather constraints from many program points — in practice an AST and a separate solving pass. Deduction needs neither: the one expression to the right of = is sufficient, and it can be consumed the instant it is parsed. *What this work adds to Selfie is type deduction, not Hindley–Milner inference*, and that is exactly why it fits a single pass with no AST. We keep the customary word "inference" in the keyword's informal name (as C++ programmers colloquially do), but the mechanism is deduction; Section 2 makes its one-directional, initialiser-only nature explicit.

1.4 Related Work

Local type deduction from an initialiser is not new; what is specific here is doing it in a single pass without an AST. The classic compiler pipeline [7] separates parsing, semantic analysis, and code generation, and resolves types on an explicit syntax tree — the setting in which Hindley–Milner inference [2] is normally described. Production languages show that deduction needs far less: C++ `auto` [3] and C23 `auto` [4] deduce a local's type from its initialiser, and Go's `:=` does the same, all specified as a one-directional, declaration-local rule rather than whole-program inference. Single-pass, AST-free compilers in the Wirth tradition [5], of which Selfie's is a descendant, already thread type information through recursive-descent parsing while emitting code directly; production single-pass C compilers such as TCC [9] do the same, tracking each operand's type on a value stack rather than on a persisting tree. What all of these share, and what the classic pipeline relaxes, is a *declaration-before-use* rule: because a name's type is fixed before

it is read, type information never flows backward — precisely the property that makes initialiser-local deduction expressible in one pass. Selfie itself has been extended before in this spirit, for instance by a conservative garbage collector added as a bachelor project [6]. This work occupies the intersection: it shows that the initialiser-local deduction of C++/Go/C23 needs none of the AST machinery of the classic pipeline, because a Wirth-style single pass already computes the required type as a by-product of code generation.

1.5 The Target Architecture

Although deduction needs no knowledge of the target machine, the implementation does: an `auto` declaration must reserve storage for its variable, and in Selfie that storage lives on the call stack. This section reviews the parts of Selfie's target needed to follow the implementation.

1.5.1 RISC-U and C*

Selfie [1] compiles C* to RISC-U. C* is a strict subset of C with only two types — the unsigned 64-bit integer `uint64_t` and the pointer `uint64_t*` — no floating point, and no composite types. RISC-U is a tiny subset of the RISC-V instruction set [8] with 14 instructions (`lui`, `addi`, `add`, `sub`, `mul`, `divu`, `remu`, `sltu`, `ld`, `sd`, `beq`, `jal`, `jalr`, `ecall`). Instructions use the 32-bit RISC-V encoding and are four-byte aligned, whereas registers and data words are 64-bit, so a data address is always a multiple of eight bytes. The two-type system is central to this work: it is why `auto` has only two possible answers and why there is no promotion lattice to search.

1.5.2 Memory

A RISC-U program runs in a byte-addressed linear address space divided into four segments: code, data, heap, and stack (Figure 1). Code and data are static and fixed at compile time; the heap and stack are dynamic. The heap sits just above the static data and grows upward (it backs `malloc`); the stack sits at the top of a 4 GB (2^{32}-byte) virtual address space and grows downward, even though registers are 64-bit. The boundary between the static and dynamic parts is the program break (Equation 1).

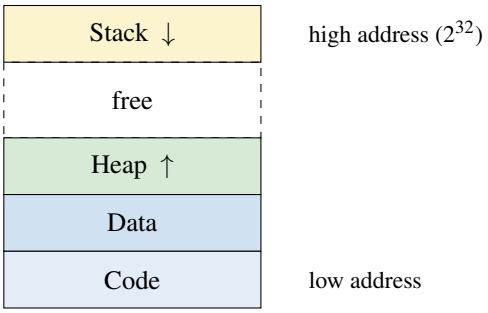

Figure 1: Memory segment order. Code and data are static; heap and stack are dynamic and grow towards each other.

$$program\_break = 65536 + code\_segment\_size + data\_segment\_size \tag{1}$$

Here the constant 65536 is the fixed 64 KB start address of the code segment; *code_segment_size* and *data_segment_size* are known once the whole source has been compiled, and the break separates the static segments from the dynamic heap and stack.

1.5.3 Function Calling Convention and the Stack Frame

Local variables live in the call stack. On a call, the caller pushes any live temporaries and the arguments; the callee's prologue then saves the return address and the caller's frame pointer, sets up its own frame

pointer in register `S0`, and reserves space for the callee's local variables by lowering the stack pointer `SP`. Locals are addressed at negative offsets from the frame pointer. Figure 2 shows the layout; the shaded region marks the contribution `auto` adds, explained in Section 2.5.

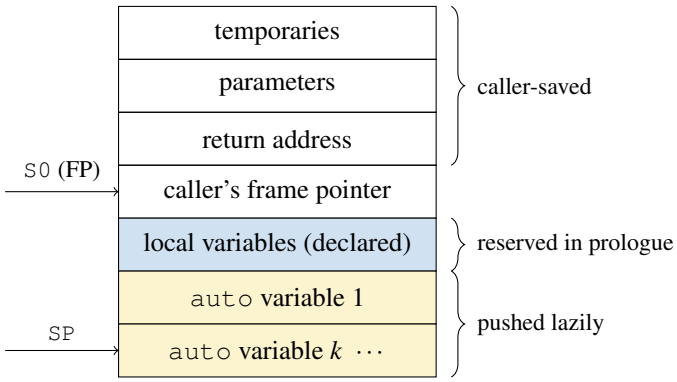

Figure 2: Call stack frame. Declared locals are reserved once in the prologue; each `auto` variable pushes its own word below them, lazily, as it is compiled.

1.5.4 The Single-Pass Compiler

Selfie's compiler is a single-pass recursive-descent parser: scanner, parser, and code emitter advance in lock-step (Figure 3). As tokens are consumed, RISC-U code is emitted immediately. There is no explicit AST; the syntax "tree" exists only implicitly in the call stack of the parser. The key property this work exploits is that the expression-compiling functions *return a type tag*. As `compile_expression()` and the precedence levels below it (`compile_arithmetic()`, `compile_term()`, `compile_factor()`) emit code that computes a value into a temporary register, each returns the type (`UINT64_T` or `UINT64STAR_T`) of that value (Figure 4).

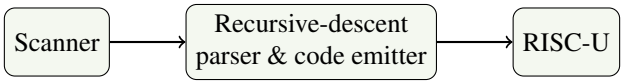

Figure 3: Selfie's single pass: scanner, parser, and emitter advance in lock-step, producing RISC-U directly with no intermediate AST.

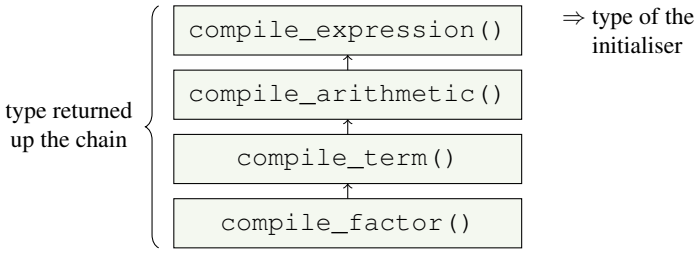

Figure 4: Each precedence level returns the type tag of the value it compiled. The type `auto` needs is the return value of `compile_expression()`.

1.5.5 Availability

This architecture is implemented by the Selfie project [1], an educational platform for teaching the design and implementation of compilers, emulators, and hypervisors. Selfie is a single self-compiling file of roughly 12 000 lines. The design of its compiler is inspired by the Oberon compiler of Niklaus Wirth [5]. Several features have been added to Selfie as bachelor projects in this same single-pass spirit, for instance a conservative garbage collector [6]; the present work follows that tradition.

2 The `auto` Implementation

This section describes the actual implementation. As with the rest of Selfie, the goal is to keep it as small as possible: `auto` is realised by reusing existing machinery rather than adding new subsystems.

2.1 The Interface

To the C* programmer, `auto` is a drop-in for a type in a local declaration with an initialiser. The extension keeps C* a strict superset: every valid C* program still compiles unchanged, and no new type keyword is introduced. An `auto` declaration is an ordinary statement inside any procedure body, including `main` (Listing 1).

Listing 1: A complete program using `auto`

```
uint64_t factorial(uint64_t n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

uint64_t main() {
    auto n   = 5;              // uint64_t
    auto res = factorial(n);   // uint64_t  -- callee return type
    auto p   = malloc(8);      // uint64_t* -- malloc result
    *p = res;
    auto v   = *p;             // uint64_t  -- dereference
    return v - 120;            // 0 on success
}
```

2.2 Scanner: the `SYM_AUTO` Symbol

The scanner gains a single keyword. The identifier `auto` is recognised and mapped to a new symbol `SYM_AUTO`, exactly as the existing type keywords map to `SYM_UINT64`. This is the only change to the scanner.

2.3 The Two Deducible Types

`auto` deduces one of Selfie's two existing types (Table 1). It adds no types, widths, or conversions: the deduced type is precisely what Selfie's expression compiler already produces for the initialiser. Because there is no second numeric type, there is no promotion rule and no implicit conversion to specify. The pointer rules are inherited verbatim from C* (Table 2); `auto` merely reports their result.

Table 1: The two types `auto` can deduce

Type	Internal tag	Example	Deduced Type
`uint64_t`	`UINT64_T`	`auto y = 1;`	`uint64_t`
`uint64_t*`	`UINT64STAR_T`	`auto p = malloc(8);`	`uint64_t*`

2.4 Parsing an `auto` Declaration

The core of the feature is the new function `compile_auto_declaration()`. At a high level it parses `auto identifier = expression ;`, reads back the type the expression compiler returns, reserves a stack slot, and records the variable in the local symbol table (Listing 2).

Table 2: Pointer type rules (unchanged from Selfie)

Expression	Condition	Result Type
`auto v = *p`	`p : uint64_t*`	`uint64_t`
`auto q = p + n`	`p : uint64_t*, n : uint64_t`	`uint64_t*`
`auto d = p - q`	`p, q : uint64_t*`	`uint64_t`
`auto r = malloc(n)`	`n : uint64_t`	`uint64_t*`
`p + q`	`p, q : uint64_t*`	**Error**

Listing 2: Pseudocode for `compile_auto_declaration`

```
compile_auto_declaration():
        consume 'auto'
        name = identifier
        consume identifier
        consume '='
        type = compile_expression()    // type Selfie already returns
        push one word onto the stack    // lazy allocation
        record (name, type) in local symbol table
        consume ';'
```

The actual C* implementation is shown in Listing 3. There is no type computation of its own: the single line `inferred_type = compile_expression()` captures the type the parser already produced (Figure 4). This is why no AST and no separate pass are required. (The name `inferred_type` follows Selfie's existing style; the mechanism is strictly deduction, per Section 1.3.1.)

Listing 3: The core of the feature in `selfie.c`

```
1  void compile_auto_declaration() {
2    char* var_name; uint64_t inferred_type;
3    get_symbol();                          // consume 'auto'
4    var_name = identifier;
5    get_symbol();                          // consume identifier
6    get_expected_symbol(SYM_ASSIGN);       // consume '='
7
8    inferred_type = compile_expression(); // type Selfie already returns
9
10   emit_addi(REG_SP, REG_SP, -WORDSIZE); // lazy stack allocation
11   emit_store(REG_SP, 0, current_temporary());
12   tfree(1);
13
14   number_of_auto_variable_bytes = number_of_auto_variable_bytes +
       WORDSIZE;
15   create_symbol_table_entry(LOCAL_TABLE, var_name, line_number,
16     VARIABLE, inferred_type, 0, -number_of_auto_variable_bytes);
17
18   get_expected_symbol(SYM_SEMICOLON);
19 }
```

2.5 Lazy Stack Allocation

A declared local in C* is reserved once, in the prologue, because the parser counts all declared locals before emitting any statements. An `auto` variable cannot be counted that way: it is introduced by a *statement*, and in a single pass the number of `auto` statements in a body is not known until they have been parsed. The implementation therefore allocates *lazily*: each `auto` declaration pushes its own word

onto the stack at the point it is compiled (`emit_addi(REG_SP, REG_SP, -WORDSIZE)` followed by a store of the initialiser value). These words accumulate below the declared locals, in the shaded region of Figure 2. The variable's symbol-table offset is `-number_of_auto_variable_bytes`, a running counter that starts at the declared-locals size and grows by one word per `auto` declaration.

2.6 Statement Dispatch and Frame Size

Two further changes integrate the new function. First, `compile_statement()` gains one branch so that an `auto` declaration is accepted wherever a statement is allowed. Second, `compile_procedure()` initialises the running counter `number_of_auto_variable_bytes` to the declared-locals size and passes the combined frame size to the epilogue. Because Selfie's epilogue restores `SP` directly from the frame pointer whenever the frame is non-empty, the exact number of pushed `auto` words never has to be tracked for correctness — the lazily pushed words are reclaimed together with the rest of the frame. With the scanner keyword, this is the full extent of the change: three functions (`compile_auto_declaration`, `compile_statement`, `compile_procedure`) plus `SYM_AUTO`.

2.7 Error Handling for Free

Because deduction reuses Selfie's expression compiler, most error cases are rejected by Selfie's existing diagnostics with no new code; only the void initialiser needs a single explicit check.

- `auto x = undefined;` — the identifier is in no symbol table, so Selfie reports an undeclared identifier.
- `auto x = y;` where `y` is declared later — in a single pass `y` is not yet visible; this is what makes forward and circular `auto` impossible.
- `auto e = p + q;` where both are pointers — Selfie already rejects pointer + pointer.
- `auto x = f();` where `f` returns `void` — there is no value to bind, so `auto` rejects the void initialiser explicitly.

3 `auto` in Selfie

Selfie's defining requirement is self-referentiality: the compiler must compile its own source. This section examines `auto` from that angle.

3.1 Self-Compilation

Because the `auto` extension keeps C* a strict superset, the unmodified `selfie.c` still self-compiles. Running `./selfie -c selfie.c -o selfie.m` produces a RISC-U binary of **43 624** instructions and **14 496** bytes of data — a 188 992-byte loaded image, since RISC-U uses the 32-bit RISC-V encoding ($43\,624 \times 4 + 14\,496 = 188\,992$) — which executes correctly under the mipster emulator. Adding `auto` grows the compiler only by the handful of instructions needed for the new function and the statement branch.

3.2 Self-Application: an `auto`-ised Selfie

Self-compilation above exercises `auto` only as a language addition, not as one the compiler's own source uses. To dogfood the feature on the compiler itself, the local declarations in `selfie.c` were rewritten to `auto` wherever the deduced type is provably identical under both the GCC bootstrap and Selfie's self-compilation. The rewrite is *cast-free*: a declaration is folded only when (a) the variable's first use is an ordinary top-level assignment, (b) the right-hand side is exactly one function call whose return type is `uint64_t`, `uint64_t*`, or `char*`, and (c) the call arguments do not reference the variable itself. The deduced type then comes directly from the callee's return type, so no cast is needed to pin it down.

This folds **79** of `selfie.c`'s **327** local declarations into `auto`; the remaining 248 stay classic declarations, which C*'s two-phase procedure grammar (a declaration block followed by a statement block) requires in any case (the categories are quantified in Section 5.4). Sites such as `auto i = 0;` or `auto p = malloc(8);` are deliberately *not* folded: GCC deduces a 32-bit `int` for the bare literal and an untyped `void*` for `malloc` (whose dereference is a hard error), so neither would survive the bootstrap even though Selfie alone would accept it. Pinning those sites instead requires an explicit cast (`auto i = (uint64_t) 0;`), which compiles but is not idiomatic. The `auto`-ised compiler translates its own source to a **43 672**-instruction, 189 184-byte image and reaches a self-self fixpoint (Section 5.1).

3.3 Boot Levels

Selfie reaches a fixpoint over boot levels: the GCC-built `selfie` (level 0) compiles `selfie.c` to a binary (level 1) that, run on mipster, compiles `selfie.c` again (level 2). Since `auto` is implemented purely in the compiler front end and emits ordinary RISC-U, it is available identically at every level. The self-application of Section 3.2 is what makes this non-trivial: the level-1 binary is now itself the product of compiling `auto` code, yet it goes on to reproduce a byte-identical level-2 binary.

4 Analysis

4.1 Cost per Declaration

Deduction adds no algorithmic cost. Compiling the initialiser is work Selfie performs for any assignment; `auto` merely keeps the type tag that ordinary compilation discards. Beyond that, an `auto` declaration performs a constant amount of additional work: one symbol-table insertion and the emission of two instructions (the `addi` that lowers `SP` and the `sd` that stores the initial value). The feature is therefore $O(1)$ per declaration on top of compiling the initialiser, with no global analysis at any point. Its scope is correspondingly narrow by design: `auto` applies to local declarations in statement position and deduces one of C*'s two types from a direct initialiser, nothing more.

4.2 Why No AST or Separate Pass is Needed

The contrast with Hindley–Milner inference (Section 1.3.1) is the crux. Inference must retain enough of the program to gather and solve constraints spanning many use sites, which in practice means an AST and a solving pass. Deduction's information flow is local and one-directional: the type is a by-product of code generation that the single pass already computes at the declaration site. Capturing it requires nothing that outlives the current statement. The restrictions on the feature — direct initialiser only, two types, no forward or circular references — are not weaknesses but the precise boundary of what a single pass makes available.

5 Experiments

5.1 Self-Compilation Fixpoint

The strongest correctness criterion for a self-hosting compiler is a self-self fixpoint: a compiler that, used to recompile its own source, produces a binary identical to itself. We verified this for the `auto`-ised `selfie.c` of Section 3.2. The GCC-built compiler compiles the `auto`-ised source to `selfie0.m`; running `selfie0.m` on mipster to recompile the same source yields `selfie1.m`. The two binaries are byte-identical (194 720 bytes each; 43 672 instructions; 189 184-byte loaded image), confirming the fixpoint. The project's `make self-self-check` target reproduces this comparison and exits successfully.

5.2 Autograder Results

Two autograder targets are relevant (Table 3). The `self-compile` target confirms self-compilation still succeeds (all checks pass). The dedicated `auto-type-inference` target exercises the feature with positive and negative programs; each positive program both compiles *and* returns exit code 0, so an incorrect deduction would change the result — the suite validates the deduced type, not merely acceptance of the syntax.

Table 3: Autograder `auto-type-inference` suite: all 14 programs behave as required.

Program	Checks	Result
`auto-fint`	`uint64_t` from integer literal	compiles, exit 0
`auto-fint-pointer`	`uint64_t*` from typed pointer	compiles, exit 0
`auto-fstring`	`uint64_t*` from string literal	compiles, exit 0
`auto-deref`	`uint64_t` from `*p`	compiles, exit 0
`auto-comparison`	`uint64_t` from a comparison	compiles, exit 0
`auto-nested`	nested `auto` declarations	compiles, exit 0
`auto-function-return`	`auto` from a call's return type	compiles, exit 0
`auto-pointer-arith`	`uint64_t*` from pointer arithmetic	compiles, exit 0
`proposal-example`	the specification's worked example	compiles, exit 0
`proposal-example-exact`	byte-exact variant of the example	compiles, exit 0
`invalid-undefined-var`	`auto x = undefined;`	rejected
`invalid-circular-dep`	`auto` referring to itself	rejected
`invalid-fstring-add`	pointer + pointer initialiser	rejected
`invalid-void-auto`	`auto` bound to a `void` call	rejected

5.3 Comparison with Selfie

Set against baseline Selfie, the result is deliberately unremarkable: the single-pass architecture, the absence of an AST and of a separate semantic pass, the two-type system, the RISC-U target, self-compilation, and the self-self fixpoint are all unchanged. The only differences are the feature itself (`auto` deduction, previously absent) and the three functions changed to implement it.

5.4 Fold Rate and Code-Size Impact on `selfie.c`

The self-application of Section 3.2 folds 79 of the 327 local declarations in `selfie.c`. A static scan classifies why the other 248 are *not* folded (Table 4): most simply do not have a single typed call as their initialiser, and in the next-largest group the initialiser *is* a call but the variable's first assignment is conditional, so its first use is not the top-level assignment the cast-free rule requires. None of the 248 is a case the deducer *could not* type; they are excluded by the conservative, cast-free folding predicate, not by a limitation of `auto`. The `malloc`/`void*` hazard of Section 3.2 is a rule-level illustration, not a driver of these counts: `selfie.c`'s single allocation site already carries an explicit cast, so `void*` folds do not arise in practice.

Table 4: Why the 248 non-folded declarations are left classic. A static scan classifies 236; the remaining twelve have multi-line declarations the scan does not track.

Reason not folded	Count
Initialiser is not a single call (literal, constant, copy, or arithmetic)	129
Initialiser is a call, but the first assignment is conditional (not top-level)	53
Initialiser already contains, or would require, an explicit cast	30
Call whose return type is outside the cast-free set	24
Classified	**236**

The code-size cost of *using* `auto` is negligible: the baseline (no `auto` in the source) self-compiles to

43 624 instructions, the `auto`-ised source to 43 672 — 48 instructions and 192 bytes across the whole compiler (+0.11%), consistent with the constant, two-instruction per-declaration cost of Section 4.1.

6 Conclusion

This work shows how cheaply initialiser-driven `auto` integrates into a single-pass, AST-free compiler. Initialiser-local deduction is single-pass by nature — C++ and Go demonstrate as much — so the contribution is not that it is possible but that in Selfie it is nearly free: the type an `auto` variable needs is a by-product of code generation the single pass already computes and would otherwise discard, and capturing it in three functions is enough, with no AST, no separate semantic pass, and no change to the code generator. Concretely: **RQ1** yes — deduction adds no AST and no separate pass; **RQ2** the type is read back from `compile_expression()`'s existing return value and the slot is pushed lazily onto the stack, a change localised to three functions plus one scanner keyword; **RQ3** yes — the compiler still self-compiles and, with its own source `auto`-ised, still reaches a byte-identical self-self fixpoint. The feature's restrictions are not weaknesses but the precise boundary of what a single pass makes available, and its error cases fall out of Selfie's existing diagnostics for free.

References

[1] Kirsch, C. M. *Selfie and the Basics*. Onward! 2017 (SPLASH), ACM, 2017. http://selfie.cs.uni-salzburg.at

[2] Milner, R. *A Theory of Type Polymorphism in Programming*. Journal of Computer and System Sciences, 17(3):348–375, 1978.

[3] ISO/IEC 14882:2020. *Programming Languages — C++*. International Organization for Standardization, 2020. Clause 9.2.9.7 [dcl.spec.auto].

[4] ISO/IEC 9899:2024. *Programming Languages — C* (C23). International Organization for Standardization, 2024.

[5] Wirth, N. *Compiler Construction*. Addison-Wesley, 1996.

[6] Bachinger, G. *Conservative Garbage Collection in Kernel and Mutator Space*. Bachelor's thesis, University of Salzburg, 2020.

[7] Aho, A. V., Lam, M. S., Sethi, R., Ullman, J. D. *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson, 2006.

[8] Waterman, A., Asanović, K. (eds.). *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*. RISC-V Foundation, 2019.

[9] Bellard, F. *TCC: Tiny C Compiler — Reference Documentation*. https://bellard.org/tcc/, 2017.